Load Testing and Stress Testing

prove capacity before launch day does it for you

Watch the 4-min video

In one line: load tests confirm targets at expected traffic, stress tests find the breaking point, soak tests catch slow leaks, and spike tests check autoscaling; the traffic model and environment must be realistic, pass criteria (p95/p99, errors) are set beforehand, every resource is watched, and the tool must avoid coordinated omission.

1 "It can handle 5,000 requests per second"

How do you know? Launch day is the worst time to find out. **Load testing** proves capacity before real users do.

2 Four kinds of test

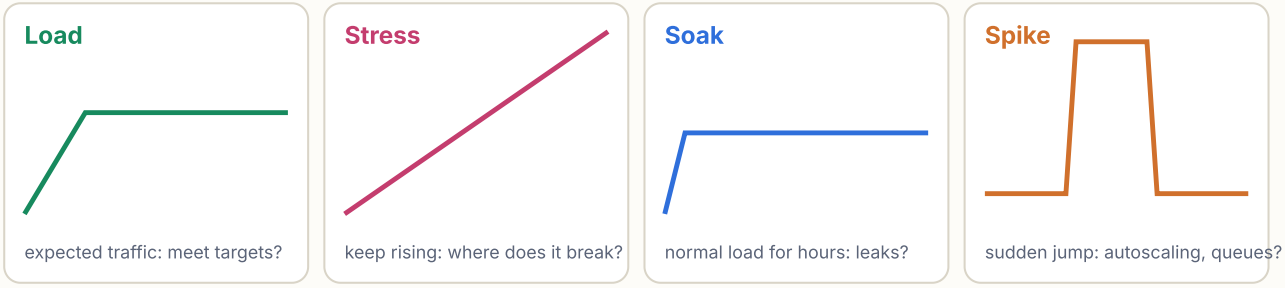


Figure 1. Four test shapes, four questions. A good plan uses more than one.

- **Load test**: expected traffic, to confirm latency and error targets.
- **Stress test**: keep increasing load beyond normal, to find the breaking point and see *how* the system fails (gracefully with 503s, or by collapsing).
- **Soak test**: normal load for hours, to catch memory leaks, connection leaks and slow degradation.
- **Spike test**: jump traffic suddenly, to check autoscaling, queues and load shedding (lessons 10 and 14).

3 The traffic model matters most

- A realistic **mix** of endpoints, e.g. 80% reads and 20% writes, weighted like production.
- Realistic **data**: many different users and products, so caches aren't unrealistically hot.
- **Think time** between user actions.
- A **production-like environment**: same instance sizes, same database size and indexes, same limits.

4 Tools

Locust (Python: simulated users are Python classes, a natural fit for Python teams), **k6** (JavaScript scripts, very efficient, good thresholds), and **JMeter** (older Java tool with a GUI). Run load generators from separate machines, and make sure *they* aren't the bottleneck (watch their CPU).

5 Measure the right things

Throughput (requests per second), latency **percentiles** (p95, p99, not averages), error rate, and server-side resources (CPU, memory, connection pools, the database and caches), so you know which saturates first. Set **pass criteria before the test**, not after.

WORKED EXAMPLE: PROVING 5,000 REQUESTS PER SECOND

A Locust test ramps to 5,000 requests per second over 10 minutes with a realistic mix of search, product views and checkout. Targets: p99 < 300 ms, errors < 0.1%. Result: p99 is **240 ms** at 5,000 per second. Pushing further, the database connection pool saturates at about **7,000**. Now you know the capacity, the headroom, and the next bottleneck.

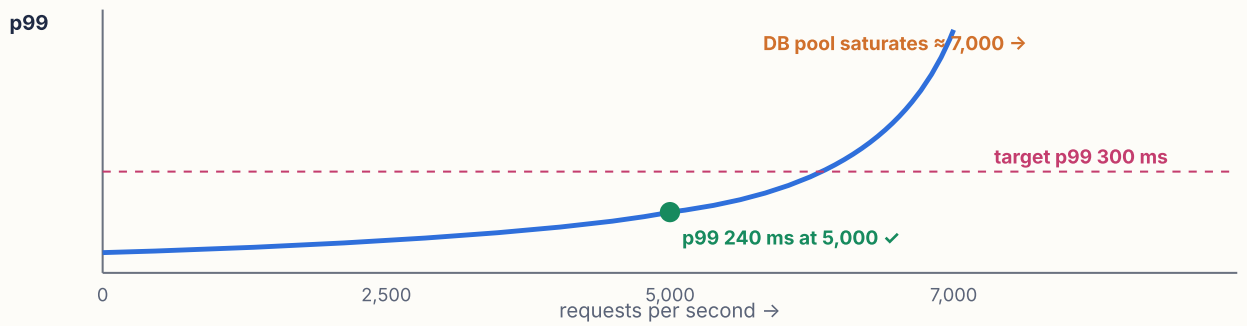


Figure 2. Latency against load: under target at 5,000 per second, with the knee near 7,000 where the connection pool saturates.

6 Make it routine, and avoid coordinated omission

Run a short load test in CI for critical paths to catch regressions, and soak tests before major releases. Beware **coordinated omission**: if the load tool waits for slow responses before sending the next request (a closed loop), it sends fewer requests exactly when the system is slow, and under-reports latency. Use tools that keep sending at a fixed rate (k6 constant-arrival-rate, wrk2).



Figure 3. Coordinated omission: a closed-loop tool hides stalls; an open-loop tool measures them.

COMMON MISTAKES

- Testing one endpoint with one user ID: everything is cached.
- A tiny staging database, so queries look fast.
- Reporting average latency.
- Load generators maxed out and reported as server capacity.

7 Key terms

Load test: expected traffic vs targets

Stress test: beyond normal, to the breaking point

Soak test: normal load for hours

Think time: pause between simulated user actions

Spike test: sudden jump in traffic

Coordinated omission: a closed-loop tool hiding slow periods

8 Interview questions

Q1. How do you design a load test? BASIC

Define the goal and pass criteria, model realistic traffic and data, ramp load in a production-like environment, and measure throughput, latency percentiles, errors and resource saturation.

Q2. What is a soak test for? BASIC

Running normal load for hours, to find memory leaks, resource exhaustion and slow degradation.

Q3. Your load test shows 20 ms p99 but production is 400 ms. Why might that be? SCENARIO

Unrealistic test: hot caches from repeated IDs, a small database, missing downstream calls, a different traffic mix, or coordinated omission in the tool.

Q4. How do you find the capacity of a service? SCENARIO

A stress test that ramps load in steps while watching p99, errors and resources; capacity is the highest rate that still meets the targets, and the first saturated resource is the next bottleneck.

Q5. What is coordinated omission? TRICKY

When a load tool waits for each response before sending the next, a slow period reduces the number of requests sent, so the slow period is under-represented in the results; fixed-rate (open-loop) generation fixes it.

Q6. Why set pass criteria before the test? TRICKY

So results can't be rationalised afterwards, and the test can fail a CI pipeline automatically when p99 or error rates regress.

Next: SD-J-012 · Queueing Theory Fundamentals backendroadmap.com